MODULE *protocol*

Zcash $P2P$ protocol specification following $ZIP-204$.

Models the connection lifecycle between peers: handshake (version/*verack*), keepalive (ping/pong), block synchronization (*inv*/*getheaders*/headers/*getdata*/block), and timeout-based disconnection.

Communication uses a message consumption model — each peer has a per-connection *FIFO inbox* and all decisions are based on message payloads only, matching the information constraints of a real network.

The spec verifies one liveness property (all peers eventually reach the same block height) and several safety invariants derived from $ZIP-204$ bounds.

EXTENDS TLC, $Naturals$, $Sequences$, $FiniteSets$, $messages$

CONSTANT $InitialPeers$
CONSTANT $MaxBlock$
CONSTANT $MaxClock$
CONSTANT $DisconnectTimeout$
CONSTANT $MinPeerProtoVersion$

VARIABLES $nodes$, $clock$

$vars \triangleq \langle nodes, clock \rangle$

See *README* for an explanation of symmetry reduction.

$Symmetry \triangleq Permutations(InitialPeers)$

For each initial peer construct a set of all other peers.

$OtherPeers \triangleq [n \in InitialPeers \mapsto InitialPeers \setminus \{n\}]$

$$
\begin{array}{l}
Init \triangleq \\
\quad \land\ clock = 0 \\
\quad \land\ \exists\, blockset \in [InitialPeers \to (1 \,..\, MaxBlock)] : \\
\qquad nodes = [i \in InitialPeers \mapsto [\\
\qquad\quad inbox \mapsto [j \in OtherPeers[i] \mapsto \langle\rangle], \\
\qquad\quad conn \mapsto [j \in OtherPeers[i] \mapsto \text{“init”}], \\
\qquad\quad ping\_nonce \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad last\_recv\_at \mapsto [j \in OtherPeers[i] \mapsto 0], \\
\qquad\quad blocks \mapsto 1 \,..\, blockset[i] \\
\qquad]]
\end{array}
$$

— Handshake —

n sends its version to m. The message lands in m's *inbox*.

$$
\begin{array}{l}
SendVersion \triangleq \\
\quad \exists\, n \in InitialPeers : \\
\qquad \exists\, m \in OtherPeers[n] :
\end{array}
$$

$\land\ nodes[n].conn[m] = \text{“init”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![m].inbox[n] = Append(@,\ MakeVersion(n,\ m,\ nodes[n].blocks)),$
$\quad ![n].conn[m] = \text{“version\_sent”}]$
$\land \text{ UNCHANGED } \langle clock \rangle$

n receives m's version from its *inbox*. Validates the version field:
valid ($\geq MinPeerProtoVersion$) $\rightarrow$ send *verack*, transition to established
invalid ($< MinPeerProtoVersion$) $\rightarrow$ send reject, reset to *init*

$RecvVersion \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] = \text{“version\_sent”}$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“version”}$
$\land \text{ LET } msg \triangleq Head(nodes[n].inbox[m])$
IN
$\lor\ \land\ msg.payload.version \geq MinPeerProtoVersion$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].inbox[m] = Tail(@),$
$\quad ![m].inbox[n] = Append(@,\ MakeVerack),$
$\quad ![n].conn[m] = \text{“established”},$
$\quad ![n].last\_recv\_at[m] = clock]$
$\lor\ \land\ msg.payload.version < MinPeerProtoVersion$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].inbox[m] = Tail(@),$
$\quad ![m].inbox[n] = \langle MakeReject(\text{“version”})\rangle,$
$\quad ![n].conn[m] = \text{“init”},$
$\quad ![n].ping\_nonce[m] = 0,$
$\quad ![n].last\_recv\_at[m] = clock]$
$\land \text{ UNCHANGED } \langle clock \rangle$

n receives a *verack* from m. If n is still in *version_sent*, this completes the handshake. If n already advanced past established (async), just consume the message.

$RecvVerack \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] \notin \{\text{“init”}\}$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“verack”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].inbox[m] = Tail(@),$
$\quad ![n].conn[m] = \text{IF } @ = \text{“version\_sent”} \text{ THEN “established” ELSE } @,$
$\quad ![n].last\_recv\_at[m] = clock]$
$\land \text{ UNCHANGED } \langle clock \rangle$

n receives an unexpected version from m while already past the handshake.
This models a reconnecting peer whose old connection was torn down unilaterally.
n resets to *init*, matching Zebra's *DuplicateHandshake* disconnect behavior.

$$
\begin{aligned}
&\mathit{RecvVersionReset} \triangleq \\
&\quad \exists\, n \in \mathit{InitialPeers} : \\
&\qquad \exists\, m \in \mathit{OtherPeers}[n] : \\
&\qquad\quad \land\ \mathit{nodes}[n].\mathit{conn}[m] \notin \{\text{“init”},\ \text{“version\_sent”}\} \\
&\qquad\quad \land\ \mathit{Len}(\mathit{nodes}[n].\mathit{inbox}[m]) > 0 \\
&\qquad\quad \land\ \mathit{Head}(\mathit{nodes}[n].\mathit{inbox}[m]).\mathit{header}.\mathit{command} = \text{“version”} \\
&\qquad\quad \land\ \mathit{nodes}' = [\mathit{nodes}\ \text{EXCEPT} \\
&\qquad\qquad\qquad ![n].\mathit{inbox}[m] = \mathit{Tail}(@), \\
&\qquad\qquad\qquad ![n].\mathit{conn}[m] = \text{“init”}, \\
&\qquad\qquad\qquad ![n].\mathit{ping\_nonce}[m] = 0, \\
&\qquad\qquad\qquad ![n].\mathit{last\_recv\_at}[m] = \mathit{clock}] \\
&\qquad\quad \land\ \text{UNCHANGED}\ \langle \mathit{clock} \rangle
\end{aligned}
$$

n discards a non-version message from m while waiting for the handshake.
After a unilateral disconnect, stale messages from the old connection may still
be in the *inbox*. Matches Zebra's behavior of ignoring non-version messages
during the handshake phase.

$$
\begin{aligned}
&\mathit{DiscardStaleMessage} \triangleq \\
&\quad \exists\, n \in \mathit{InitialPeers} : \\
&\qquad \exists\, m \in \mathit{OtherPeers}[n] : \\
&\qquad\quad \land\ \mathit{nodes}[n].\mathit{conn}[m] = \text{“version\_sent”} \\
&\qquad\quad \land\ \mathit{Len}(\mathit{nodes}[n].\mathit{inbox}[m]) > 0 \\
&\qquad\quad \land\ \mathit{Head}(\mathit{nodes}[n].\mathit{inbox}[m]).\mathit{header}.\mathit{command} \notin \{\text{“version”},\ \text{“verack”},\ \text{“reject”}\} \\
&\qquad\quad \land\ \mathit{nodes}' = [\mathit{nodes}\ \text{EXCEPT} \\
&\qquad\qquad\qquad ![n].\mathit{inbox}[m] = \mathit{Tail}(@)] \\
&\qquad\quad \land\ \text{UNCHANGED}\ \langle \mathit{clock} \rangle
\end{aligned}
$$

n receives a reject from m, resetting to *init*.

$$
\begin{aligned}
&\mathit{RecvReject} \triangleq \\
&\quad \exists\, n \in \mathit{InitialPeers} : \\
&\qquad \exists\, m \in \mathit{OtherPeers}[n] : \\
&\qquad\quad \land\ \mathit{Len}(\mathit{nodes}[n].\mathit{inbox}[m]) > 0 \\
&\qquad\quad \land\ \mathit{Head}(\mathit{nodes}[n].\mathit{inbox}[m]).\mathit{header}.\mathit{command} = \text{“reject”} \\
&\qquad\quad \land\ \mathit{nodes}' = [\mathit{nodes}\ \text{EXCEPT} \\
&\qquad\qquad\qquad ![n].\mathit{inbox}[m] = \mathit{Tail}(@), \\
&\qquad\qquad\qquad ![n].\mathit{conn}[m] = \text{“init”}, \\
&\qquad\qquad\qquad ![n].\mathit{ping\_nonce}[m] = 0, \\
&\qquad\qquad\qquad ![n].\mathit{last\_recv\_at}[m] = \mathit{clock}] \\
&\qquad\quad \land\ \text{UNCHANGED}\ \langle \mathit{clock} \rangle
\end{aligned}
$$

— *Keepalive* —

n sends a ping to m when idle.

$SendPing \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] \notin \{\text{“init”},\ \text{“version\_sent”}\}$
$\land\ nodes[n].last\_recv\_at[m] \leq clock - 3$
$\land\ nodes[n].ping\_nonce[m] = 0$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$![n].inbox[n] = Append(@,\ MakePing),$
$![n].ping\_nonce[m] = clock]$
$\land \text{ UNCHANGED } \langle clock \rangle$

n receives a ping from m and immediately replies with pong.
$RecvPing \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] \notin \{\text{“init”},\ \text{“version\_sent”}\}$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“ping”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$![n].inbox[m] = Tail(@),$
$![m].inbox[n] = Append(@,\ MakePong),$
$![n].last\_recv\_at[m] = clock]$
$\land \text{ UNCHANGED } \langle clock \rangle$

n receives a pong from m, completing the keepalive round-trip.
$RecvPong \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].ping\_nonce[m] \neq 0$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“pong”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$![n].inbox[m] = Tail(@),$
$![n].ping\_nonce[m] = 0,$
$![n].last\_recv\_at[m] = clock]$
$\land \text{ UNCHANGED } \langle clock \rangle$

—— Block sync ——

n announces its inventory to m after handshake.
$SendInv \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] = \text{“established”}$
$\land\ nodes[n].blocks \neq \{\}$
$\land\ nodes' = [nodes \text{ EXCEPT}$

```
                    ![m].inbox[n]          = Append(@, MakeInv(nodes[n].blocks)),
                    ![n].conn[m]           = "inv_sent",
                    ![n].last_recv_at[m] = clock]
          ∧ UNCHANGED ⟨clock⟩
```

n receives an *inv* from m. Both peers must have sent their invs first (n is *inv_sent*). Inspects the *payload* to decide: sync or already caught up.

```
RecvInv ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∧ nodes[n].conn[m] = "inv_sent"
      ∧ Len(nodes[n].inbox[m]) > 0
      ∧ Head(nodes[n].inbox[m]).header.command = "inv"
      ∧ LET msg ≜ Head(nodes[n].inbox[m])
            advertised ≜ msg.payload.inventory[1].hash
        IN
         ∨ ∧ advertised > Cardinality(nodes[n].blocks)
           ∧ nodes' = [nodes EXCEPT
                    ![n].inbox[m]          = Tail(@),
                    ![m].inbox[n]          = Append(@, MakeGetHeaders(nodes[n].blocks)),
                    ![n].conn[m]           = "getheaders_sent",
                    ![n].last_recv_at[m] = clock]
         ∨ ∧ advertised ≤ Cardinality(nodes[n].blocks)
           ∧ nodes' = [nodes EXCEPT
                    ![n].inbox[m] = Tail(@),
                    ![n].conn[m]  = "synced"]
      ∧ UNCHANGED ⟨clock⟩
```

n receives a *getheaders* request from m and responds with its own headers.

```
RecvGetHeaders ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∧ nodes[n].conn[m] ∉ {"init", "version_sent"}
      ∧ Len(nodes[n].inbox[m]) > 0
      ∧ Head(nodes[n].inbox[m]).header.command = "getheaders"
      ∧ nodes' = [nodes EXCEPT
                ![n].inbox[m]          = Tail(@),
                ![m].inbox[n]          = Append(@, MakeHeaders(nodes[n].blocks)),
                ![n].last_recv_at[m] = clock]
      ∧ UNCHANGED ⟨clock⟩
```

n receives headers from m (response to its *getheaders*), sends *getdata*.

```
RecvHeaders ≜
  ∃ n ∈ InitialPeers :
    ∃ m ∈ OtherPeers[n] :
      ∧ nodes[n].conn[m] = "getheaders_sent"
```

$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“headers”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].inbox[m] = Tail(@),$
$\quad ![m].inbox[n] = Append(@,\ MakeGetData(nodes[n].blocks)),$
$\quad ![n].conn[m] = \text{“getdata\_sent”},$
$\quad ![n].last\_recv\_at[m] = clock]$
$\land\ \text{UNCHANGED } \langle clock \rangle$

n receives a *getdata* request from m and responds with a block.

$RecvGetData \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] \notin \{\text{“init”},\ \text{“version\_sent”}\}$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“getdata”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$
$\quad ![n].inbox[m] = Tail(@),$
$\quad ![m].inbox[n] = Append(@,\ MakeBlock(nodes[n].blocks)),$
$\quad ![n].last\_recv\_at[m] = clock]$
$\land\ \text{UNCHANGED } \langle clock \rangle$

n receives a block from m, adds it to its local chain.

$RecvBlock \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] = \text{“getdata\_sent”}$
$\land\ Len(nodes[n].inbox[m]) > 0$
$\land\ Head(nodes[n].inbox[m]).header.command = \text{“block”}$
$\land\ \text{LET } newBlocks \triangleq nodes[n].blocks \cup \{Cardinality(nodes[n].blocks) + 1\}$
$\quad msg \triangleq Head(nodes[n].inbox[m])$
$\quad \text{IN } nodes' = [nodes \text{ EXCEPT}$
$\quad\quad ![n].inbox[m] = Tail(@),$
$\quad\quad ![n].last\_recv\_at[m] = clock,$
$\quad\quad ![n].blocks = newBlocks,$
$\quad\quad ![n].conn[m] = \text{IF } Cardinality(newBlocks) < msg.payload.prev\_block$
$\quad\quad\quad \text{THEN “block\_received”}$
$\quad\quad\quad \text{ELSE “synced”}]$
$\land\ \text{UNCHANGED } \langle clock \rangle$

n has received a block but still needs more — sends another *getdata*.

$SendGetData \triangleq$
$\exists\, n \in InitialPeers :$
$\exists\, m \in OtherPeers[n] :$
$\land\ nodes[n].conn[m] = \text{“block\_received”}$
$\land\ nodes' = [nodes \text{ EXCEPT}$

$$
\begin{aligned}
&!\,[m].inbox[n] &&= Append(@,\ MakeGetData(nodes[n].blocks)),\\
&!\,[n].conn[m] &&= \text{“getdata\_sent”},\\
&!\,[n].last\_recv\_at[m] &&= clock]\\
&\land \text{UNCHANGED } \langle clock \rangle
\end{aligned}
$$

— Disconnection —

Unilateral disconnect: models *TCP* FIN — only the detecting side resets.
The remote peer's *inbox* from n is also cleared (the *TCP* pipe is broken,
so messages in transit are lost), but m's connection state is unchanged —
m will independently detect the timeout and disconnect on its own.
This matches Zebra's implementation where each side detects idleness independently.

$$
\begin{aligned}
&Disconnect \triangleq\\
&\quad \exists\, n \in InitialPeers :\\
&\qquad \exists\, m \in OtherPeers[n] :\\
&\qquad\quad \land\ nodes[n].conn[m] \notin \{\text{“init”}\}\\
&\qquad\quad \land\ clock - nodes[n].last\_recv\_at[m] > DisconnectTimeout\\
&\qquad\quad \land\ nodes' = [nodes \text{ EXCEPT}\\
&\qquad\qquad\qquad !\,[n].inbox[m] &&= \langle\rangle,\\
&\qquad\qquad\qquad !\,[m].inbox[n] &&= \langle\rangle,\\
&\qquad\qquad\qquad !\,[n].conn[m] &&= \text{“init”},\\
&\qquad\qquad\qquad !\,[n].ping\_nonce[m] &&= 0,\\
&\qquad\qquad\qquad !\,[n].last\_recv\_at[m] &&= clock]\\
&\qquad\quad \land \text{UNCHANGED } \langle clock \rangle
\end{aligned}
$$

$$
\begin{aligned}
&Tick \triangleq\\
&\quad \land\ clock < MaxClock\\
&\quad \land\ clock' = clock + 1\\
&\quad \land \text{UNCHANGED } \langle nodes \rangle
\end{aligned}
$$

$$
\begin{aligned}
&Next \triangleq\\
&\quad \lor\ Tick\\
&\quad \lor\ SendVersion\\
&\quad \lor\ RecvVersion\\
&\quad \lor\ RecvVersionReset\\
&\quad \lor\ DiscardStaleMessage\\
&\quad \lor\ RecvVerack\\
&\quad \lor\ RecvReject\\
&\quad \lor\ SendPing\\
&\quad \lor\ RecvPing\\
&\quad \lor\ RecvPong\\
&\quad \lor\ SendInv\\
&\quad \lor\ RecvInv\\
&\quad \lor\ RecvGetHeaders
\end{aligned}
$$

$\lor\ RecvHeaders$
$\lor\ RecvGetData$
$\lor\ RecvBlock$
$\lor\ SendGetData$
$\lor\ Disconnect$

$Spec \triangleq$
$Init$
$\land\ \Box[Next]_{vars}$
$\land\ \mathrm{WF}_{vars}(Next)$

Liveness property: all peers eventually reach the same block height.
$EventualConsensus \triangleq \Diamond\forall\, i, j \in InitialPeers : nodes[i].blocks = nodes[j].blocks$

$ZIP-0204$ invariants: safety properties checked at every reachable state.

inv and *getdata* inventory vectors must not exceed 50,000 entries ($ZIP-0204$ §4).
$InvCountBounded \triangleq$
$\forall\, n \in InitialPeers :$
$\forall\, m \in OtherPeers[n] :$
$\forall\, i \in 1\,..\,Len(nodes[n].inbox[m]) :$
$nodes[n].inbox[m][i].header.command = \text{“inv”}$
$\Rightarrow nodes[n].inbox[m][i].payload.count \leq 50000$

$GetDataCountBounded \triangleq$
$\forall\, n \in InitialPeers :$
$\forall\, m \in OtherPeers[n] :$
$\forall\, i \in 1\,..\,Len(nodes[n].inbox[m]) :$
$nodes[n].inbox[m][i].header.command = \text{“getdata”}$
$\Rightarrow nodes[n].inbox[m][i].payload.count \leq 50000$

headers messages must not carry more than 160 block headers ($ZIP-0204$ §4).
$HeadersCountBounded \triangleq$
$\forall\, n \in InitialPeers :$
$\forall\, m \in OtherPeers[n] :$
$\forall\, i \in 1\,..\,Len(nodes[n].inbox[m]) :$
$nodes[n].inbox[m][i].header.command = \text{“headers”}$
$\Rightarrow nodes[n].inbox[m][i].payload.count \leq 160$

Peers must speak at least $MinPeerProtoVersion$ ($ZIP-0204$ §3).
$VersionBounded \triangleq$
$\forall\, n \in InitialPeers :$
$\forall\, m \in OtherPeers[n] :$
$\forall\, i \in 1\,..\,Len(nodes[n].inbox[m]) :$
$nodes[n].inbox[m][i].header.command = \text{“version”}$
$\Rightarrow nodes[n].inbox[m][i].payload.version \geq MinPeerProtoVersion$

A non-zero ping nonce implies the connection is past the handshake.

$$PingOnEstablished \triangleq$$
$$\forall\, n \in InitialPeers :$$
$$\quad \forall\, m \in OtherPeers[n] :$$
$$\quad\quad nodes[n].ping\_nonce[m] \neq 0$$
$$\quad\quad \Rightarrow nodes[n].conn[m] \notin \{\text{“init”},\ \text{“version\_sent”}\}$$

A peer only enters sync states when it actually has fewer blocks than its partner.
NOTE: uses $\leq$ (not $<$) because with message consumption, m could gain blocks from a third peer between when n enters a sync state and when we check.

$$SyncDirection \triangleq$$
$$\forall\, n \in InitialPeers :$$
$$\quad \forall\, m \in OtherPeers[n] :$$
$$\quad\quad nodes[n].conn[m] \in \{\text{“getheaders\_sent”},\ \text{“getdata\_sent”},\ \text{“block\_received”}\}$$
$$\quad\quad \Rightarrow Cardinality(nodes[n].blocks) \leq Cardinality(nodes[m].blocks)$$